Chapter 11: Packs, Suites, Metadata, and Production Use

Download this chapter as a PDF for offline reading:
Chapter-11-Packs-Suites-Metadata-and-Production-Use.pdf

So far you have written individual queries. In production those queries live in *packs*, are selected by *suites*, carry *metadata*, and must be tested before they are trusted. This chapter shows how to organize that work.

Packs and Dependencies

A CodeQL pack is a versioned collection of queries and libraries. It is defined by a `qlpack.yml` file.

Minimal example (see `examples/chapter11/qlpack.yml`):

```yaml
name: mustlearn/example-queries
version: 0.1.0
dependencies:
  codeql/javascript-all: "*"
extractor: javascript
```

Common pack commands

```
codeql pack init my-org/my-queries
codeql pack add codeql/javascript-all
codeql pack install
codeql pack publish          # optional, for sharing
```

A practical workflow:

1. Initialize the pack.
2. Add the language pack(s) you depend on.
3. Place your `.ql` files in a `queries/` directory (or any layout you prefer).
4. Install dependencies.
5. Compile and run against a local database to verify.

Packs can depend on other packs. The standard libraries are themselves packs (`codeql/javascript-all`, `codeql/python-all`, etc.). Your custom model pack from Chapter 10 would be published and then added as a dependency here.

Suites and Metadata

Suites

A suite (`.qls` file) selects which queries from a pack should run. The simplest suite includes everything:

```
- description: Example suite
- queries: .
  from: codeql/javascript-all
```

(See `examples/chapter11/default.qls`.)

You can also exclude queries, include only certain tags, or compose multiple suites. Suites are how you turn a large pack into a focused security suite, a quality suite, or a custom suite for a particular team.

Metadata

Every production query should carry the QLDoc metadata introduced in Chapter 6:

```
/**
 * @name ...
 * @description ...
 * @kind problem          // or path-problem, metric, etc.
 * @id org/query-name
 * @tags security
 * @problem.severity error
 * @precision high
 * @security-severity 8.0
 */
```

The `@id` must be unique. The severity, precision, and security-severity values drive filtering in GitHub code scanning and in `codeql database analyze`. Incomplete metadata makes a query harder to operate in production.

Testing and Production Checklist

Testing

Before a query is considered production-ready:

- Compile it (`codeql query compile`).
- Run it against one or more representative databases.
- Confirm expected results appear and unexpected results do not.
- For path queries, inspect a few paths in the VS Code extension.
- If the query uses custom models, verify those models with the tests from Chapter 10.

Production checklist

- ☐ Query has complete metadata (`@name`, `@description`, `@kind`, `@id`, `@tags`, severity, precision).
- ☐ Query lives in a pack with a correct `qlpack.yml`.
- ☐ Dependencies are declared and installed.
- ☐ A suite selects the queries you intend to run.
- ☐ The query has been tested on real code.
- ☐ Results are reviewed for false positives before enabling in CI or code scanning.
- ☐ (Optional) The pack is versioned and published so others can depend on it.

Next Steps

You can now package queries, select them with suites, attach the metadata that production systems expect, and verify them before they run on every commit. The next chapter shows how to integrate those packs into GitHub Actions and other CI systems.

All examples referenced above live in the companion repository:
github.com/rod-trent/MustLearnCodeQL
under `examples/chapter11/`.

Must Learn CodeQL — Rod Trent
Companion repository: github.com/rod-trent/MustLearnCodeQL